ET1556 Introduction to scientific work
September 2023



Technical Debt in Requirements Engineering

Naga Pujitha Pitta

Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden

This project report is submitted to the Department of Mathematics and Natural Sciences at Blekinge Institute of Technology in partial fulfilment of the requirements for the course ET1556.

Contact Information:
Author(s):
Naga Pujitha Pitta
E-mail: napi23@student.bth.se

University advisor:
Irina gertsovich
Department of Mathematics and Natural Sciences

Dept. of Mathematics and Natural Sciences
Blekinge Institute of Technology
SE–371 79 Karlskrona, Sweden

Internet : www.bth.se
Phone : +46 455 38 50 00
Fax : +46 455 38 50 57

Abstract

Context: Technical debt in requirements engineering refers to the shortcuts or suboptimal decisions made during the initial stages of software development, especially in defining requirements. These shortcuts may lead to problems later like increased development effort or maintenance challenges, akin to financial debt with interest. Addressing technical debt early is crucial to ensure a robust and maintainable software system.
Objectives: The study aims to provide an in-depth understanding of Technical Debt by investigating challenges related to Requirements Engineering. The main objective is to explore typical roles and responsibilities of TD in RE and to find out how practitioner's challenges are faced and handled by the software professionals.
Methods: A systematic mapping study (SMS), which analyzes the quantity and type of evidence that is present in the research, provides a broad review of the research area on a certain issue. A systematic mapping study is often regarded as a good place to start for additional research [10]. Since there are no appropriate research publications on the difficulties faced by practitioners with technical debt. The purpose of the survey is to learn about the practitioner's experiences with carrying out software system development. We'll have the survey questions ready for SMS. The survey was created using the SMS's results as a guide. In order to comprehend technical debt in requirements engineering and extend the results from the SMS that enable us to locate the new or unknown difficulties, surveys assist in collecting data from a bigger audience by taking into account different domains and industries. Due to the large population, the survey not only produces more accurate data but also offers broader study perspectives.
Results: Gaining a better understanding of technical debt in requirement engineering. We will get to know how the practitioners classify or categorize the challenges faced when developing a product in software systems and we will have a better understanding of the needs of practitioners and managed by the practitioners in software systems.
Conclusions: we hereby conclude that this project would be very helpful Since requirements are important in software, our study will try to define requirement technical debt and how to manage it.
Keywords: Technical debt, Requirements engineering, Systematic mapping study(SMS), Requirements technical debt.

List of Figures

4.1 Challenges of TD in RE xx
4.2 Roles of Respondents xx

Contents

Abstract **iii**

1 Introduction **ix**
1.1 Aim and Objectives x
1.2 Research Questions x
1.3 Research Gap xi
1.4 Ethical, societal and sustainability aspects xi

2 Related Work **xiii**

3 Method **xv**
3.1 Systematic Mapping Study xv
3.2 Survey xvi

4 Results and Analysis **xix**
4.1 Analysis xix

5 Discussions **xxiii**
5.1 RQ1 How do software professionals understand the notion of TD in RE? xxiii
5.2 RQ2 What are the challenges of TD in RE? xxiii
5.3 What new perspectives do these results add to the body of current research? xxv
5.4 Limitations xxv

6 Conclusions and Future Work **xxvii**
6.1 Conclusion xxvii
6.2 Future Work xxviii

References **xxix**

Chapter 1

Introduction

Even in carefully planned initiatives, obstacles like market pressure can have a negative impact on the end product in software development [16]. In this situation, the development team must find alternate means of completing the tasks in the short term, frequently without the support of the customer to complete the software before the deadline, resource constraints, or pressure from the market itself, all of which may have a negative long-term effect on the software [4]. In this situation, the project's quality problems may be discovered before, during, or after its completion. Therefore, tasks that have been compromised must be upgraded at some time throughout the project; otherwise, Technical Debt (TD) [3] may emerge.

Technical Debt:
TD refers to problems caused by pending or inefficient tasks [8]. Taking these actions can produce short-term benefits, such as an increase in productivity, but they can also hinder the development of the project in the long run. Thus, TD includes items usually controlled in a software project, such as unimplemented features. It also addresses less obvious problems like outdated documentation and code smells [2].
"Technical debt" is the effect of emphasizing speed over accuracy. For instance, a vehicle to travel sometimes people constantly decide to put off that investment for months or even years in order to save the money they actually need as a result they acquire a car loan that means they'll pay more over time but they'll get what they need right away.

Types of technical debt(TD):
1. planned or intentional
2. unplanned or unintentional

1. planed or intentional : It is a technical debt taken deliberately as a strategic tool this type makes a well-informed choice to incur technical debt [16].
2. unplanned or unintentional : Not strategic outcome of bad performance unplanned technical debt occurs as a result of poor techniques lack of data etc [16].

Requirement engineering :
The process to gather the software requirements from customers, and analyze and document them is known as requirement engineering.

Technical Debt in Requirements Engineering:
"Technical debt in requirements engineering" is a metaphorical way of describing the long-term costs and problems that can arise when software requirements are not properly defined, clarified, or documented early in a project. It's like borrowing money and promising to pay it back later, but with interest. If you rush through or neglect requirements, it can lead to complications, rework, and additional effort down the road as developers struggle to adapt the software to what's needed. In essence, it's the hidden cost of taking shortcuts in the early stages of software development.

Requirements engineering is one of the disciplines of software engineering that seeks to use and analyze strategies and actions to gather, specify, and document a set of requirements that respond to the needs of stakeholders with a high-quality result. The software's behavior, functionality, limitations, and all other properties are all described in the requirements. Nevertheless, this is a challenging step that stakeholders (analysts, consumers, and developers) sometimes overlook [19].

As the research progressed, the notion was expanded to include other phases of software and software development, such as requirements engineering [10], while the initial focus of TD was on coding operations [3]. According to [5], errors that increase the prevalence of TD in software projects are brought on by insufficient requirements elicitation or analysis. In this situation, the technical debt of requirements may happen unintentionally if the requirements engineers lack experience and the necessary skills to carry out technical and lengthy procedures, or it may happen intentionally if a conscious decision is made not to be rigorous in the elicitation process [16].

1.1 Aim and Objectives

The main "aim" is to provide an in-depth of Technical Debt by investigating and challenges related to Requirements Engineering. The main "objectives" understanding of the project are exploring typical roles and responsibilities of technical debt (TD) in requirements engineering (RE) involves understanding how TDs facilitate the management, analysis, and communication of software requirements within a project.

1.2 Research Questions

1) How do software professionals understand the notion of Technical Debt (TD) in Requirements Engineering (RE)? In the formal work process, many people describe many ideas on technical debt but we want what software professionals think and talk about technical debt in requirement engineering.
2) What are the challenges in TD in RE? Understanding how practitioners handle challenges faced by software professionals in Software Engineering (SE), involves researching and identifying the common issues and difficulties experienced in the field, as well as examining the strategies and solutions employed by software professionals to address these challenges effectively.

1.3 Research Gap

Technical debt in software design and code was the primary area of research [22]. Academic researchers don't frequently look into requirement debt. We don't yet have a specific definition for requirement debt, and it's unclear how to handle it. Despite the fact that Ernst [5] was the first to discuss requirement technical debt, the description is relatively brief and leaves out key information. We need more details on this idea, such as the various causes it might occur and how to categorize it. Yli-Huumo believe that there are currently no effective tools for managing requirement technical debt. As a result, we can see that the research on this subject hasn't been done in great detail, and there isn't a consensus on what it is or how to handle it. Given the importance of requirements in software, our study will attempt to identify technical debt associated with requirements and explain how to manage it is the gap in this project.

1.4 Ethical, societal and sustainability aspects

1. Ethical Aspect:
Technical debt in requirements engineering has significant ethical implications. When organizations accumulate technical debt knowingly or unknowingly, they may be compromising the quality, reliability, and security of the software systems they deliver to users. This can lead to ethical dilemmas, as it potentially jeopardizes user trust and data privacy. For instance, if requirements are poorly documented or inadequately tested due to rushed development, it could result in vulnerabilities that expose user data to potential breaches.
2. Societal Aspect:
The societal impact of technical debt in requirements engineering extends beyond individual projects and can affect entire communities. When software systems riddled with technical debt are deployed in critical sectors such as healthcare, finance, or transportation, they may lead to system failures or errors that can harm individuals or even pose risks to public safety.
3. Sustainability Aspect:
Sustainability considerations encompass not only environmental aspects but also the long-term viability of software systems. Accumulating technical debt in requirements engineering can be unsustainable in the sense that it leads to increased maintenance costs, reduced agility, and decreased software longevity. This hinders the ability to adapt to changing user needs or evolving technological landscapes. Sustainable software development practices require managing technical debt effectively to ensure that software systems remain relevant, maintainable, and eco-friendly over time.

Understanding and addressing technical debt in requirements engineering from these ethical, societal, and sustainability perspectives is critical to delivering software that aligns with responsible and sustainable development practices, upholding ethical standards, and safeguarding the well-being of both individuals and society at large.

Chapter 2

Related Work

The goal of [18] was to offer a specific description of this kind of TD by investigating the state of the TD of requirements as it stands at the time. They carried out a survey and an SMS(Systematic mapping study) to accomplish this. Ten measurement methodologies were found among the findings, along with advice from software experts on how to find this TD. Finally, they realized that the way academia and industry approach this TD differs, and they pushed both to work together. Since some of the material in this work is related to the aforementioned study above, it is anticipated that it will support it. The expenses or efforts associated with requirements technical debt were not, however, measured using metrics.

Last but not least, the study by [9] looked into the methods for prioritizing TD that have been suggested in the fields of software engineering research and industry. They carried out an systematic mapping study(SMS) to do this, and the final product contained 44 main studies. They noted among the findings that the research on TD prioritizing is in its early stages and that there is no agreement on the parameters that are crucial and how to measure them. They will then suggest a mind map that software experts can use to prioritize TD tasks.

Because it strives to provide information that assists in carrying out a certain stage of the management process of TD assertively, the cited study is comparable to the current proposal of this work. However, by providing varied data for the research setting, this work seeks to add to the first two stages of identification and measurement.

The cited works and the current article are similar since they both look for approaches to comprehend and control TD. But in contrast to the publications listed, this study used both human and automatic search methods. As a complement, we also use the snowballing method. As a result, there are more research sources to take into account. On the one hand, this current work is comparable to studies by [11]and [18], despite the fact that they have not addressed the reasons for their emergence and metrics for measurement methodologies. In order to aid in the management of TD, [15] and [1] concentrated on TD in general. Lenarduzzi et al.'s paper from 2021 examined a particular stage (prioritization) of the TD management process, varied by omitting to discuss the evidence pertaining to its identification and measurement.

The technique put out by [6]served as the foundation for the systematic mapping study(SMS) carried out in this study. The authors claim that a pre-defined systematic approach is used in conjunction with protocols and processes to locate and analyze the evidence found in Primary Studies. SMSs attempt to link primary studies in terms of their findings and look into whether these findings are congruent with the aims. In order to synthesis the evidence, SMSs must take into account its strength [7].

The authors say that this study was the initial step in the development of new technologies that aid in the management of TD as well as in the establishment and validation of a taxonomy on the many types of TD. Although it is regarded as a pertinent study in the field, it does not yet provide evidence to help measure TD, particularly through the use of metrics, so that repayment decisions can be based on this kind of evidence, which is one of the goals of this work. Finally, "Systematic mapping study (SMS) is to provide a broad review of the research area on a particular topic and identify the quantity and what type of evidence is available in the research".

Chapter 3

Method

Experiments [21], surveys [14], and case studies [17] are three often used methods of inquiry in the field of computer science engineering. Case studies provide to greater awareness of societal, organizational, and human phenomena [17]. The only method stated above that is appropriate for our investigation is the survey.

The inability to execute experiments because not all the components are known previously is one of the reasons we did not choose an experiment for this study [21]. The systematic Literature review (SLR) is not appropriate for our research since it rigorously extracts the acquired results and verifies the quality of the evidence from each paper. A systematic literature review is a comprehensive and structured approach to gather, evaluate, and synthesize existing research on a specific topic. It involves predefined search criteria, rigorous selection processes, and critical analysis to provide an unbiased summary of the current state of knowledge in a particular field. This method aims to minimize bias and ensure that all relevant studies are considered to make informed conclusions or identify gaps in research and because there aren't any appropriate related research articles to do SLR on [20].

3.1 Systematic Mapping Study

"Systematic mapping study" is to provide a broad review of the research area on a particular topic and identify the quantity and what type of evidence is available in the research" [12] [13]. A system mapping study is a research method used to visually represent and analyze complex systems. It involves identifying the key components, relationships, and interactions within a system to gain a better understanding of how it functions. This approach often uses diagrams or models to illustrate the system's structure and dynamics, helping researchers identify areas for improvement or intervention. System mapping studies are valuable for addressing complex issues in fields like ecology, business, healthcare, and social sciences. A thorough mapping study is often regarded as a good place to start for additional research [20]. Since there are no appropriate research publications on the difficulties faced by practitioners with technical debt.

A systematic mapping study is a research method for identifying, categorizing, and summarizing existing literature on a certain topic. Here's a brief description of the procedure:
Establish the Research Question: Indicate the topic or research issue you wish to investigate.
Look for Related Literature: Systematically search numerous sources for relevant papers and publications, such as databases, journals, and conference proceedings.
Selection and screening: The papers were screened and chosen based on predetermined inclusion and exclusion criteria.
Data Extraction: Take crucial information from the selected publications, such as research methodologies, findings, and publishing information, and extract it.
Data Synthesis: Sort and summarize the extracted data to find patterns, trends, and gaps in the literature.
Reporting: Compile the results and findings into a complete report, which should include visual representations such as tables or charts.
Refinement: Based on responses and insights, refine the research to improve its quality and accuracy.

Remember that a systematic mapping study, rather than providing an in-depth investigation, tries to present an overview of the published literature on a specific issue.

3.2 Survey

A survey is a means of gathering information or data from a group of people, typically referred to as respondents, by asking them a series of questions. Surveys can take several forms, including online surveys, paper forms, phone interviews, and in-person interviews. A survey's purpose is to collect opinions, preferences, or factual information from a specific group of people in order to understand trends, make choices, or conduct research. Surveys can be utilized in a variety of contexts, such as market research, social science studies, customer feedback, and so on.

Using Google Forms to create a survey to examine the issues of technical debt in requirements engineering can be summarized as follows:
Create a Google Form: To create a new form, open Google Forms and click the "+ Blank" button.
Title and Description: Give your survey a clear title, such as Technical Debt in Requirements Engineering, and include a brief description of the survey's goal, which is to identify the challenges TD in RE.
Add Questions: Create questions that address technical debt challenges in requirements engineering. Multiple-choice questions, rating scales, and open-ended inquiries are among examples.
Design the Survey: Arrange your questions rationally, beginning with general inquiries and progressing to more specialized ones.
Submit the Survey: Send the survey link to your target audience by email, social media, or other methods of contact.

Gather all the Responses: Keep an eye on the responses as they come in using Google Forms.
figure out Results: Once a significant number of responses have been collected, assess the data to uncover patterns and challenges associated with technical debt in requirements engineering.
Report Findings: Present your survey findings clearly and straightforwardly, if necessary, using graphical representations.

Chapter 4

Results and Analysis

A Systematic mapping study is regarded as a good starting point for subsequent research [20]. Because there are no appropriate research publications on practitioners' technical debt concerns. We believe that this strategy is appropriate for answering RQ2 in our research. The purpose of the survey is to learn about the practitioners' experiences with software development. We will prepare the survey questionnaire by SMS. The SMS results serve as the foundation for the survey design. Surveys help collect data from a bigger audience by considering multiple areas and industries to assess technical debt in requirements engineering and extend the SMS results that allow us to locate new or previously unknown difficulties. Because the population is large, the survey not only provides more trustworthy data but also provides broader viewpoints for the study. The information is gathered by an online questionnaire distributed to various respondents.

4.1 Analysis

I conducted a poll on eight challenges: multiple interests, RE model integration, turnover, missing requirements, support for Interactive RE Tool, prioritization of needs, RE, and conflicting goals. Based on past works, I identified different challenges and created a survey questionnaire to collect data. I generated Google forms for these challenges, distributed them to various IT personnel, and gathered responses. I received comments from them, and I discovered that some difficulties are most likely addressed by IT professionals, while others are not. Some tasks received larger percentages, while others received very low percentages. I created a pie chart on the roles of respondents who responded to the survey.

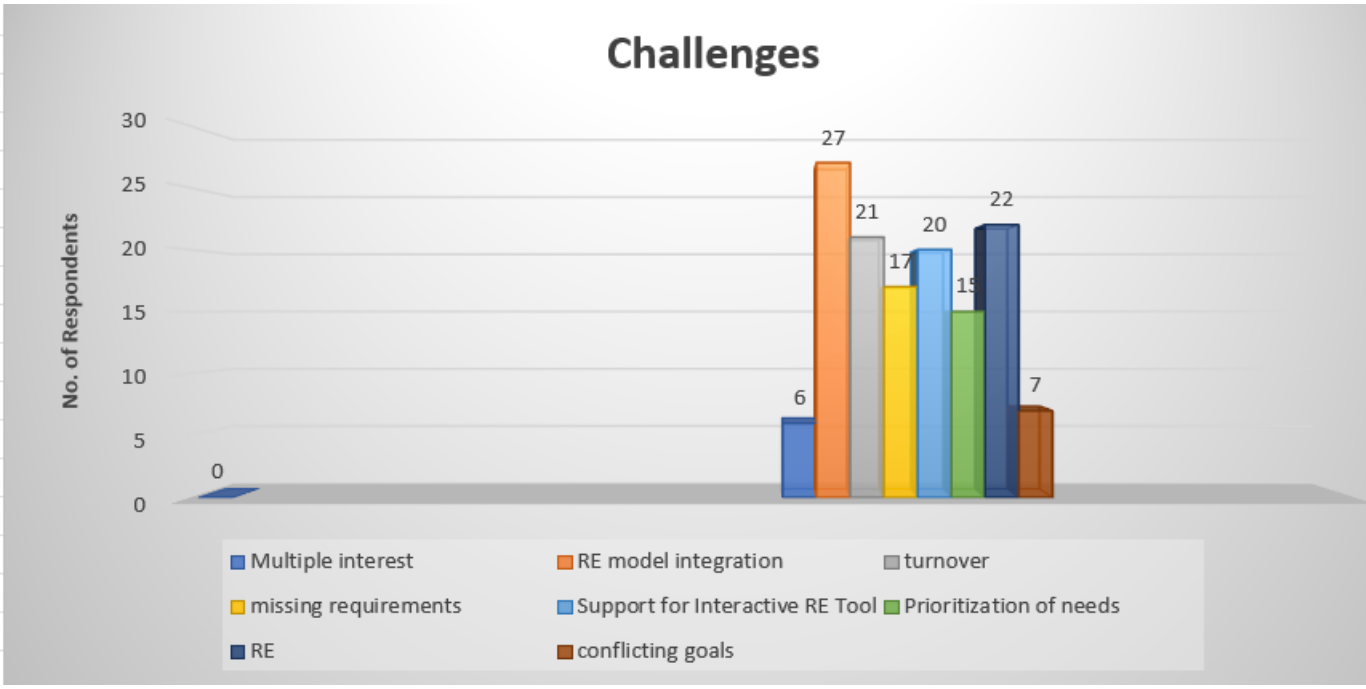


Figure 4.1: Challenges of TD in RE

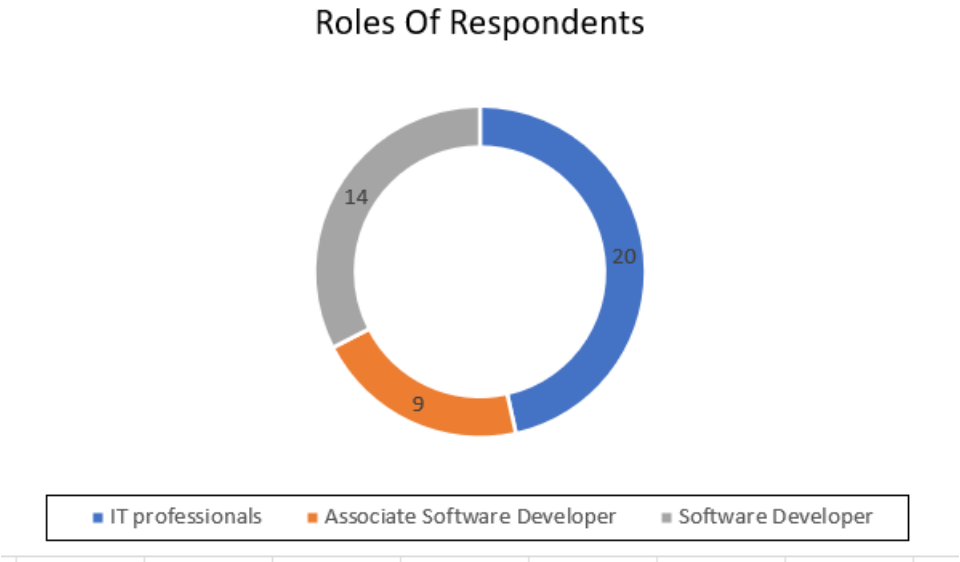


Figure 4.2: Roles of Respondents

To be more specific, I inquired about the difficulties associated with Technical Debt in Requirements Engineering. I created some challenges in Google Forms and distributed them to various people via email or other means. Some people provided numerous ideas and suggestions, while others had no notion. Furthermore, for some of the responses, respondents only provided a few words or nouns to convey their ideas, which could be hard to classify. We chose to refine the keywords and extract the basic notions of the responses in order to organize the ideas offered by the respondents.

Chapter 5

Discussions

5.1 RQ1 How do software professionals understand the notion of TD in RE?

Technical debt is a strongly embedded idea in the field of software development, and it has a substantial impact on requirements engineering. It's a metaphor for the effects of making short-term, expedient or substandard decisions, which can lead to long-term problems and additional development efforts. In the fast-paced world of software development, software professionals understand that these trade-offs are frequently essential. In practice, they recognize that Technical Debt increases over time. Developers incur this debt when they choose to cut corners, postpone critical enhancements, or use fast solutions to meet deadlines or deliver features quickly. They understand that while these shortcuts may be beneficial in the short term, they might cause issues in the long run.

To manage Technical Debt successfully, software professionals must be aware of its presence and implications. They are careful in their decision-making. They must balance the benefits of faster delivery against the long-term consequences of Technical Debt. As a result, businesses must prioritize and schedule refactoring, code changes, and quality assurance as part of their development process.In essence, technical debt in requirements engineering is viewed as a balancing act by software experts. They strive to achieve a balance between providing software rapidly to fulfill business demands and addressing Technical Debt to ensure the product is long-term and maintainable. To ensure the long-term success of software projects, it is necessary to make deliberate choices, manage the effects, and maintain a healthy equilibrium.

5.2 RQ2 What are the challenges of TD in RE?

1. RE Model Integration: Model integration refers to the process of combining multiple requirement models and specifications into a single uniform framework in the context of Technical Debt in Requirements Engineering. Failure to do so can lead to Technical Debt, as inconsistencies and gaps between models can collect over time, making it more difficult to maintain, comprehend, and evolve the system. Resolving these integration challenges can be expensive and time-consuming, emphasizing the significance of proactively maintaining and harmonizing requirements models to avoid Technical Debt accumulation during the development process.

2. Missing Requirements: "Missing requirements" in needs Engineering relate to critical system requirements that were not sufficiently discovered, documented, or addressed during the early phases of development. Because these requirements must later be discovered, documented, and integrated, this mistake results in Technical Debt, incurring additional costs and problems. Missing requirement management is critical for avoiding long-term concerns and maintaining system quality.

3. Support for Interactive RE Tools: "Support for interactive RE tools" refers to the continuing maintenance and development necessary for tools used in interactive requirements engineering procedures in the context of Technical Debt in Requirements Engineering. Neglecting such assistance might result in Technical Debt, as these tools may become outdated, less efficient, or incompatible with changing project requirements. To keep the tools effective in facilitating collaborative and interactive requirements engineering activities, they must be updated and customized on a regular basis.

4. Prioritization of needs: The method of allocating importance and urgency to specific requirements is referred to as "prioritization of needs" in Technical Debt within Requirements Engineering. Neglecting this prioritization can result in Technical Debt, since necessary demands may be deprioritized, potentially causing issues with system functionality or performance. Properly managing this component entails a methodical strategy to requirements based on commercial value, risk, and stakeholder input, hence lowering Technical Debt.

5. RE: "RE" stands for Requirements Engineering itself in the context of Technical Debt in Requirements Engineering. Technical debt in RE refers to the total cost and issues incurred as a result of taking shortcuts or making poor decisions throughout the requirements gathering and documentation process. This can result in difficulties like inadequate or confusing requirements, which, if not addressed, can lead to greater development effort and system complexities over time. Managing Technical Debt in RE entails keeping needs well-defined and up-to-date in order to avoid long-term issues.

6. Turnover: The quantity of interest can alter due to non-technical causes such as persons quitting projects; for example, the same TD may have a different interest if a specific developer leaves.

I chose the challenges that had been largely identified in previous articles. I discovered a total of eight challenges. According to the results of the survey, only six challenges are known, and two challenges are unknown to IT experts. Multiple interests, RE model integration, turnover, missing requirements, support for Interactive RE Tool, prioritization of needs, RE, and conflicting goals are the difficulties I've chosen for the survey. Multiple interests and conflicting goals are two less noted issues in these challenges. I don't think so, because these are also some of the primary issues of technical debt in requirements engineering, in my opinion. Because having multiple interests is one of the obstacles to TD in RE. Multiple interests are defined

as One TD that can have a bad impact on multiple aspects of software quality at the same time; each aspect can have its own level of interest. For example, "uncommented code" is harmful to both maintainability and dependability. Each of these two features has its own level of interest and likelihood of concurrence, which are difficult to predict. As a result, this is also related, and it may be one of the major issues in TD in RE. Another issue is competing objectives. "Conflicting goals" in needs Engineering relate to circumstances in which distinct stakeholders have opposing or incompatible needs and objectives. Failure to resolve these disputes might result in Technical Debt since the system may become difficult to maintain or satisfy all stakeholders. To achieve a good project conclusion, excellent communication, negotiation, and documentation are required to strike a balance and create a compromise among conflicting aims.

The major overview of this thesis's difficulty is that technological debt research began in the 1990s. Scientists focused mostly on technical debt in software design and code [22]. Academic researchers rarely investigate requirement debt. So far, we lack a clear and specific definition of requirement debt, as well as guidance on how to manage it. Even though Ernst [5] was the first to discuss Requirement Technical Debt, his formulation is relatively brief and without clarity. We need more information about this idea, such as the various causes and classifications. Yli-Huumo and colleagues [22] People believe that there are no good techniques for dealing with required technical debt. As a result, we can see that the study on this problem is lacking in-depth, and there is no clear agreement on what it is and how to deal with it. Because requirements are vital in software, our research will attempt to define requirement technical debt and how to manage it.

5.3 What new perspectives do these results add to the body of current research?

Previous studies focused mostly on Technical Debt in software design and coding, with little attention paid to requirement debt. Requirement debt is difficult to identify and manage because it lacks a defined and complete definition. Existing definitions, notably those of Ernst, are judged insufficient and do not contain the required elements to completely comprehend the notion. The reasons for the requirement debt and the methods for classifying it are unknown. There is a lack of thorough research in this area, and there is no clear consensus on what requirement debt is and how to successfully manage it. In this study, We gave a clear definition of it.

5.4 Limitations

Limited historical data, subjectivity in defining requirement debt, complexity in classifying it, resource constraints, scarcity of management tools, evolving requirements, interrelated challenges, stakeholder conflicts, and limited prior research for comparison or validation are all potential limitations on technical debt in requirements engineering.

Chapter 6

Conclusions and Future Work

6.1 Conclusion

When software development stages are delayed or not effectively implemented, a phenomenon known as TD occurs, which, if not controlled on time and in the correct manner, results in financial and operational losses. TD can be seen at several stages of the software life cycle. However, specific varieties of this phenomenon must be studied in order to aid in its management.

In conclusion, the results provide considerable evidence for the objective of this work. Thus, the main contributions of this work are: 1) Gaining a better understanding of technical debt in requirement engineering 2) We will get to know how the practitioners classify or categorize the challenges faced when developing a product in a software system 3) Having a better understanding of the needs of practitioners face and managed by the practitioners in software systems.

Our research is expected to end in a more specific and complete definition of required technical debt, providing the groundwork for future research and practical application in this field. Furthermore, our research efforts can help to categorize requirement debt, offering a formal framework for understanding and controlling this type of technical debt. our research may shed light on the many variables and reasons that contribute to requirement debt formation during the software development process by looking into the causes of requirement debt.

We have concluded that this initiative would be extremely beneficial. Because requirements are critical in software, our research will attempt to define requirement technical debt and how to manage it.

our research delves into the intricacies of TD in the context of Requirements Engineering, and the conclusion reflects the achieved results and contributions. By gaining a better understanding of TD in requirement engineering, our research sets the groundwork for future studies and practical applications in managing and controlling TD, especially in the crucial area of software requirements. Ultimately, our research aims to define the required technical debt and provide insights on how to manage it, aligning with the broader goal outlined in the aim of our project.

6.2 Future Work

1. Develop a comprehensive definition: Future study can try to produce a more thorough and universally accepted description that incorporates various dimensions and aspects of this idea, building on your efforts to define requirement technical debt.

2. Conducting comparative research: Comparative studies that examine the impact of requirement debt in comparison to other types of technical debt, such as coding debt or design debt, can provide a more comprehensive understanding of its relevance and management.

3. Carrying out longitudinal analysis: Examining the growth of requirement debt in software projects over time, as well as its influence on project success, can provide insights into its dynamics and long-term implications.

4. Development of best practices and guidelines: Future research could result in the development of best practices and standards for requirement debt management. This can assist organizations and practitioners in addressing this issue in their projects in a proactive manner.

References

[1] M. BenIdris, "Investigate, identify and estimate the technical debt: a systematic mapping study," *Available at SSRN 3606172*, 2020.

[2] N. Brown, Y. Cai, Y. Guo, R. Kazman, M. Kim, P. Kruchten, E. Lim, A. MacCormack, R. Nord, I. Ozkaya *et al.*, "Managing technical debt in software-reliant systems," in *Proceedings of the FSE/SDP workshop on Future of software engineering research*, 2010, pp. 47–52.

[3] W. Cunningham, "The wycash portfolio management system," *ACM Sigplan Oops Messenger*, vol. 4, no. 2, pp. 29–30, 1992.

[4] E. da Silva Maldonado, E. Shihab, and N. Tsantalis, "Using natural language processing to automatically detect self-admitted technical debt," *IEEE Transactions on Software Engineering*, vol. 43, no. 11, pp. 1044–1062, 2017.

[5] N. A. Ernst, "On the role of requirements in understanding and managing technical debt," in *2012 Third International Workshop on Managing Technical Debt (MTD)*. IEEE, 2012, pp. 61–64.

[6] S. Keele *et al.*, "Guidelines for performing systematic literature reviews in software engineering," 2007.

[7] B. A. Kitchenham, D. Budgen, and O. P. Brereton, "Using mapping studies as the basis for further research–a participant-observer case study," *Information and Software Technology*, vol. 53, no. 6, pp. 638–651, 2011.

[8] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *Ieee software*, vol. 29, no. 6, pp. 18–21, 2012.

[9] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. A. Fontana, "A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools," *Journal of Systems and Software*, vol. 171, p. 110827, 2021.

[10] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.

[11] R. Nascimento, E. Aranha, U. Kulesza, and M. Lucena, "Requirements smells como indicadores de má qualidade na especificação de requisitos: Um mapeamento sistemático da literatura." in *WER*, 2018.

[12] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic mapping studies in software engineering," in *12th International Conference on Evaluation and Assessment in Software Engineering (EASE) 12*, 2008, pp. 1–10.

[13] K. Petersen, S. Vakkalanka, and L. Kuzniarz, "Guidelines for conducting systematic mapping studies in software engineering: An update," *Information and software technology*, vol. 64, pp. 1–18, 2015.

[14] T. Punter, M. Ciolkowski, B. Freimut, and I. John, "Conducting on-line surveys in software engineering," in *2003 International Symposium on Empirical Software Engineering, 2003. ISESE 2003. Proceedings.* IEEE, 2003, pp. 80–88.

[15] L. F. Ribeiro, M. A. de Freitas Farias, M. G. Mendonça, and R. O. Spínola, "Decision criteria for the payment of technical debt in software projects: A systematic mapping study." *ICEIS (1)*, pp. 572–579, 2016.

[16] N. Rios, M. G. de Mendonça Neto, and R. O. Spínola, "A tertiary study on technical debt: Types, management strategies, research trends, and base information for practitioners," *Information and Software Technology*, vol. 102, pp. 117–145, 2018.

[17] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical software engineering*, vol. 14, pp. 131–164, 2009.

[18] Q. Wang and Y. Huang, "Identification and management of requirements debt: Systematic mapping study and survey," 2020.

[19] K. Wiegers and J. Beatty, *Software requirements.* Pearson Education, 2013.

[20] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in *Proceedings of the 18th international conference on evaluation and assessment in software engineering*, 2014, pp. 1–10.

[21] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in software engineering.* Springer Science & Business Media, 2012.

[22] J. Yli-Huumo, A. Maglyas, and K. Smolander, "How do software development teams manage technical debt?–an empirical study," *Journal of Systems and Software*, vol. 120, pp. 195–218, 2016.



Faculty of Engineering, Blekinge Institute of Technology, 371 79 Karlskrona, Sweden